Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность системы управления информацией

Выполнил:
Студент группы АС-68/24
Пух К. В.
Проверил:
А. А. Крощенко
доцент кафедры ИИТ,
«___»_____ 2026 г.

Брест 2026

Вариант 7

Цель работы: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации.

Задание:

Прогнозирование оттока клиентов

- Telco Customer Churn
- Задача: предсказать, уйдет ли клиент (бинарная классификация).
- Архитектура:
  - входной слой;
  - один скрытый слой с 24 нейронами (ReLU);
  - выходной слой с 1 нейроном (Sigmoid).
- Эксперимент: добавьте в модель слой Dropout с коэффициентом 0.2 после скрытого слоя и сравните результаты. Помогло ли это?

Код программы:

```
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix
)

torch.manual_seed(42)

# 1. Подготовка данных
df = pd.read_csv("Telco-Customer-Churn.csv")
print("Размер данных:", df.shape)

# ID клиента не несёт информации для модели
df = df.drop(columns=["customerID"])

# В TotalCharges пустые значения записаны пробелом -> заполняем нулём
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
df["TotalCharges"] = df["TotalCharges"].fillna(0)

# Целевая переменная: 1 — отток (Yes), 0 — остался (No)
df["Churn"] = (df["Churn"] == "Yes").astype(int)

# One-Hot Encoding категориальных признаков
X = pd.get_dummies(df.drop(columns=["Churn"]), drop_first=True, dtype=int)
y = df["Churn"].values

print("Признаков после кодирования:", X.shape[1])

# Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Стандартизация
```

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Преобразование в тензоры PyTorch
X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1)
y_test = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)

n_features = X_train.shape[1]

# 2. Архитектура нейронной сети
class MLP(nn.Module):
    def __init__(self, n_inputs, use_dropout=False):
        super().__init__()
        self.hidden = nn.Linear(n_inputs, 24)   # скрытый слой, 24 нейрона
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.2) if use_dropout else nn.Identity()
        self.output = nn.Linear(24, 1)          # выходной слой, 1 нейрон

    def forward(self, x):
        x = self.hidden(x)
        x = self.relu(x)
        x = self.dropout(x)
        # Sigmoid встроен в функцию потерь BCEWithLogitsLoss,
        # поэтому здесь возвращаются логиты
        return self.output(x)

# 3-4. Обучение модели
def train_model(model, epochs=300):
    criterion = nn.BCEWithLogitsLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    train_losses, test_losses = [], []

    for epoch in range(epochs):
        model.train()
        y_pred = model(X_train)
        loss = criterion(y_pred, y_train)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        model.eval()
        with torch.no_grad():
            test_loss = criterion(model(X_test), y_test)

        train_losses.append(loss.item())
        test_losses.append(test_loss.item())

    return train_losses, test_losses

# 5. Оценка модели
def evaluate(model):
    model.eval()
    with torch.no_grad():
        probs = torch.sigmoid(model(X_test))     # выход Sigmoid
        pred = (probs > 0.5).int().numpy().ravel()
    true = y_test.int().numpy().ravel()
    return {
        "Accuracy": accuracy_score(true, pred),
        "Precision": precision_score(true, pred),
```

```
        "Recall": recall_score(true, pred),
        "F1-score": f1_score(true, pred),
    }, confusion_matrix(true, pred)

# Модель без Dropout
torch.manual_seed(42)
model_base = MLP(n_features, use_dropout=False)
base_train, base_test = train_model(model_base)
base_metrics, base_cm = evaluate(model_base)

# Эксперимент: модель с Dropout(0.2) после скрытого слоя
torch.manual_seed(42)
model_drop = MLP(n_features, use_dropout=True)
drop_train, drop_test = train_model(model_drop)
drop_metrics, drop_cm = evaluate(model_drop)

print("\nАрхитектура модели с Dropout:")
print(model_drop)

print("\nРезультаты на тестовой выборке")
print(f"{'Метрика':<10} {'Без Dropout':>12} {'С Dropout':>10}")
for metric in base_metrics:
    print(f"{metric:<10} {base_metrics[metric]:>12.3f} "
          f"{drop_metrics[metric]:>10.3f}")

print(f"\nLoss в конце обучения (train / test):")
print(f"Без Dropout: {base_train[-1]:.4f} / {base_test[-1]:.4f}")
print(f"С Dropout:   {drop_train[-1]:.4f} / {drop_test[-1]:.4f}")

diff = drop_metrics["F1-score"] - base_metrics["F1-score"]
print(f"\nИзменение F1-score от Dropout: {diff:+.3f}")
```

Результат работы программы:

```
Размер данных: (7043, 21)
Признаков после кодирования: 30

Архитектура модели с Dropout:
MLP(
  (hidden): Linear(in_features=30, out_features=24, bias=True)
  (relu): ReLU()
  (dropout): Dropout(p=0.2, inplace=False)
  (output): Linear(in_features=24, out_features=1, bias=True)
)

Результаты на тестовой выборке
Метрика     Без Dropout  С Dropout
Accuracy          0.791      0.793
Precision         0.629      0.633
Recall            0.516      0.529
F1-score          0.567      0.576

Loss в конце обучения (train / test):
Без Dropout: 0.4078 / 0.4242
С Dropout:   0.4152 / 0.4250

Изменение F1-score от Dropout: +0.010
```

Вывод: построил, обучил и оценил многослойный перцептрон (MLP) для решения задачи классификации.